EXERCISE: AN ALGORITHM THAT SATISFIES: NUMS[I] - 10 = K

Suppose we have a list `nums` and a constant k.

You must find the first index `i` such that `nums[i] - 10 == k`. If there is no such index that satisfies the property, return `None`.

Example cases:

Case 1:
- **Inputs**: nums = [5, 20, 35, 23, 2, 5, 30, 6, 2, 3], k = 20
- **Output**: 6
- **Explanation**: nums[6] is 30, 30 - 10 = 20, and k = 20.

Case 2:
- **Inputs**: nums = [5, 2, 60, 50, 20], k = 15
- **Output**: None
- **Explanation**: There is no index i such that `nums[i] - 10 = k`.

1. Write a multi-step strategy towards solving the problem.
2. Write the algorithm of this problem.
3. Describe the upper-bound time and space complexity of your solution.

SOLUTION:

1.
   - We must make a function that accepts two parameters: `nums` and `k`.
   - Then we need to loop the `range(n)` where n is the length of `nums`, while accessing the index `i`, and for each `nums[i] - 10`, see if it is `==` to `k`.
   - If an index is found, and the property is satisfied, we immediately return the index and exit the function.
   - Otherwise, we keep looping, and if the loop ends and no index is found, we return `None`.

2.

```
def solution(nums, k):
  for i in range(len(nums)):
    if nums[i] - 10 == k:
      return i

  return None
```

3. The solution's time complexity is $O(n)$: Because the amount of instructions scale linearly ($y = ax + b$) with the input's size.
   The solution's space complexity is $O(1)$: Because we are only storing constant data regardless of input size).

EXERCISE: THE TWO SUM PROBLEM

Suppose we have a list `nums` and a constant `k`.

You must find two indices `i` and `j` such that `nums[i] + nums[j] = k` and $i \neq j$, if there are no such indices, return `None`.

Multiple solutions may be valid.

Example cases:

Case 1:
- **Inputs**: [3,2,4], k = 6
- **Outputs**: [1,2]
- **Explanation**: nums[1] + nums[2] == 6, we return [1,2] (or [2,1])

Case 2:
- **Inputs**: nums = [2,7,11,15], k = 9
- **Outputs**: [0, 1]
- **Explanation**: Because nums[0] + nums[1] == 9, we return [0, 1]

1. Write a multi-step strategy towards solving the problem.
2. Write the algorithm of this problem.
3. Describe the upper-bound time and space complexity of your solution.

SOLUTION:

1.
   - We must make a function that accepts two parameters: `nums` and `k`.
   - We will make a dict of numbers we have seen already, the keys will be numbers we've seen and the values will be indices.
   - We will loop over `nums`, accessing `i` and `nums[i]`.
   - Assuming we have `nums[i]`, `c = k - nums[i]` will be the number we need such that `nums[i] + c = k`, we will calculate that compliment `c`.
   - If the compliment is inside the dict, then we can return the index of the compliment and `i`. (In other words, if the number c we need for `c + nums[i] = k` is inside the dict, we will return the indices of it and i).

2.
```python
def solution(nums, k):
    seen = {}

    for i in range(len(nums)):
        num = nums[i]
        compliment = k - nums[i]

        if compliment in seen:
            j = seen[compliment]
            return [i, j]

        seen[num] = i

    return None
```

3. The time complexity of the algorithm is: $O(n)$
   The space complexity of the algorithm is: $O(n)$

SOLUTION:

1.
   - We must make a function that accepts two parameters: `nums` and `k`.
   - We will loop over `range(len(nums))` and access `i` and `nums[i]`.
   - For each `num` at `i`, we will loop over `range(i + 1, len(nums))`, to check the numbers in-front of `nums[i]` excluding `nums[i]` itself, from that loop we will access `j`.
   - In the previous loop, if `nums[i] + nums[j] == k`, then we can return `[i, j]`

2.
```python
def solution(nums, k):
  for i in range(len(nums)):
    for j in range(i + 1, len(nums)):
      if nums[i] + nums[j] == k:
        return [i, j]

  return None
```

3. The time complexity of the algorithm is: $O(n^2)$: The function scales quadratically with it's input.
   The space complexity of the algorithm is: $O(1)$: We are only storing constant data.

EXERCISE: APPROXIMATE PI USING MONTE CARLO METHODS

Monte Carlo methods are computational algorithms based on **repeated random sampling**, used to solve **deterministic** problems through **randomness**.

- **Area of a circle**: $a_c = \pi r^2$, where r is the radius.
- **Area of a square**: $a_t = a^2$, where a is the side-length of the square.
- **Equation of a circle**: $x^2 + y^2 = r^2$
- **Point $P(x_p, y_p)$ is inside the circle**: $x_p^2 + y_p^2 \leq r^2$
- **Point $P(x_p, y_p)$ is outside the circle**: $x_p^2 + y_p^2 > r^2$

Now, imagine a circle of radius r centered at $(0, 0)$, inscribed in a square, the square would have side-length $a = 2r$. The area of the square would then be $a_t = (2r)^2 = 4r^2$.

Now, imagine within the constraints of this square, we are randomly choosing n amount of points, and we will consider two cases:

- Points that landed in the circle
- Points that landed in the square

If you think about it, the proportion of points inside the circle approximates the proportion of the circle's area to the square's area. We will compute $\frac{a_c}{a_t}$ to get a formula for π.

$$\frac{a_c}{a_t} = \frac{\pi r^2}{4r^2} = \frac{\pi}{4}$$

Now, we will consider the amount of points that landed in the circle as the area of the circle (a_c), and the points that landed in the square as the area of the square (a_t), so we get:

$$\frac{\text{amount of points in the circle}}{\text{amount of points in the square}} \approx \frac{\pi}{4}$$

$$\pi \approx 4\left(\frac{\text{amount of points in the circle}}{\text{amount of points in the square}}\right)$$

The way we will resolve this equation using Monte Carlo is as following: for a number n, we will generate n points with random x and y coordinates, then, for each of these points, we will use the formulas to check if they landed in the circle or not, then we could resolve this equation.

Now, because `random.random()` only provides values between `0` and `1`, we will work with a quarter-circle (first-quadrant only) where: $0 \leq x \leq 1$ and $0 \leq y \leq 1$, which simplifies the process (no need to generate negative coordinates).

Here are the visuals:

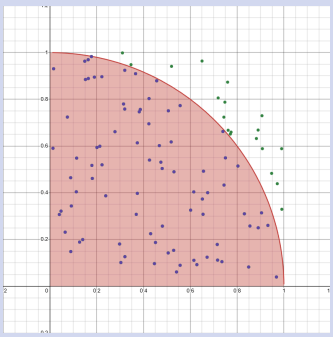

Figure 1: n = 100

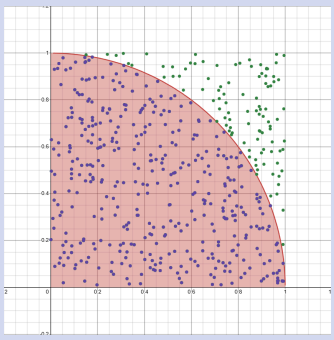

Figure 2: n = 500

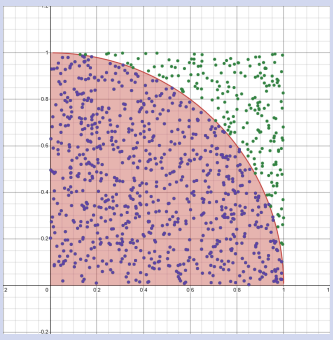

Figure 3: n = 1000

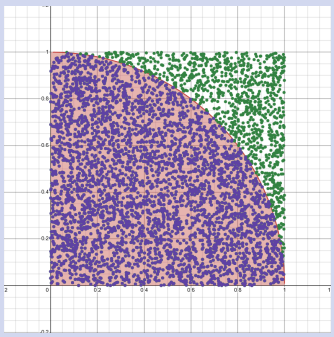

Figure 4: n = 5000

The purple points are the points *inside the circle*, and the green points are *outside the circle*.

1. Write a multi step strategy towards solving the problem.
2. Write the algorithm for this problem.
3. Describe the upper-bound time and space complexity of your solution.

SOLUTION:

1.
   - We must make a function that takes a parameter: `amount_of_points`, the number of points to be generated.
   - We will consider arbitrarly that $r = 1 \text{ unit}$.
   - We will make a variable called `points_in_circle` to accumulate how many points landed in the circle.
   - We must loop `n` times.
   - For each loop, each time, we will generate a random `x_p` and `y_p` point that lies between 0 and 1 with `random.random()` from the `random` package.
   - We will then calculate `x_p**2 + y_p**2` for that point, and check if it is less than r^2, if it is, then it is inside the circle, and we will add `+1` point to `points_in_circle`.
2.

```python
import random

def solution(amount_of_points):
    points_in_circle = 0
    r = 1

    for i in range(n):
        x_p = random.random()
        y_p = random.random()

        distance = x_p**2 + y_p**2

        if distance <= r**2:
            points_in_circle += 1

    pi = 4 * (points_in_circle / amount_of_points)
    return pi
```

3. This algorithm has a time complexity of: $O(n)$: Because instructions increase linearly with input size. And has a space complexity of: $O(1)$: Because the amount of data we are storing is not proportional and independant of the input size.

EXERCISE: APPROXIMATE PI USING AN N-SIDED HEXAGON INSCRIBED IN A CIRCLE